

LG 부트캠프 10기

프로젝트 결과보고서

피치 피치

Linux System 반 3팀 Bootcoin

강상원 김예빈 박건희 조민석

목차구성

CONTENTS COMPOSITION

1. 주제 및 결과 요약
2. 개발 목표 및 개발 결과
3. 핵심 기술
4. 결과 분석 및 기대 효과
5. 향후 연구 과제
6. 프로젝트 수행 후기

1. 주제 및 결과 요약

주제

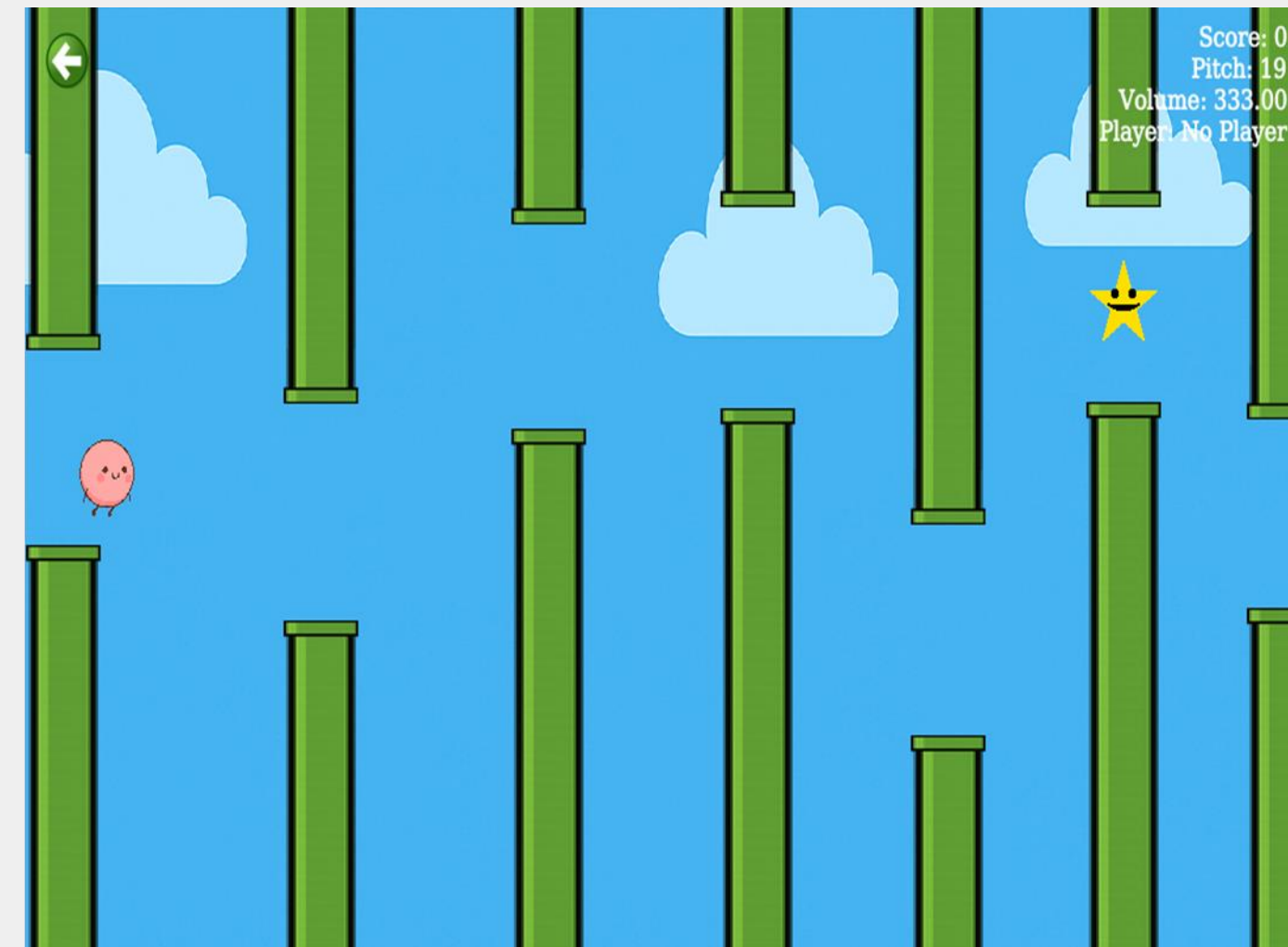
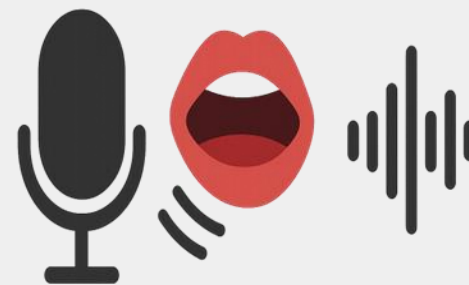
사용자의 음정을 인식해 캐릭터를 조작하는 음악 기반 게임 시스템 개발

목표

- 마이크 입력으로 음정/박자를 실시간 분석
- 싱글/멀티/노래 연습 모드 구현
- 임베디드 환경에서 독립 실행 가능한 시스템 구축

결과

- ALSA 기반 실시간 오디오 처리 및 피치 분석 성공
- Qt GUI 기반 3가지 게임 모드 완성
- UDP 멀티플레이어 통신 동기화 구현
- 커널 포팅 + Buildroot 기반 Rootfs 제작 및 자동 실행 환경 구성



2. 개발 목표 및 개발 결과

초기 개발 목표

- 사용자의 음정/박자를 실시간으로 분석해 캐릭터를 제어하는 게임 구현
- 싱글 플레이, 멀티 플레이, 노래 연습 모드 제공
- Qt 기반 GUI와 마이크 입력 기능 통합
- 임베디드 환경에서도 독립 실행 가능한 형태로 시스템 구축

실제 개발 결과

- 싱글/멀티/연습 모드 완성
- CSV 파일을 통한 노래 데이터 파싱, 다양한 곡 지원 가능
- Autocorrelation 기반 피치 검출 + MIDI 음정 매핑 구현
- UDP 멀티캐스트 통신으로 멀티플레이어 동기화 구현
- Buildroot로 커스터마이징된 Rootfs 구축, 자동 부팅/실행 지원
- 최신 커널 포팅(6.12.35) 및 ALSA, Qt5 등 필수 드라이버 적용

3. 핵심 기술: 커널 포팅

1. 임베디드 시스템 환경 구축



최신 Linux 커널 6.12.35 포팅

- Serial, Framebuffer, Ethernet, ALSA 등 필수 드라이버 설정



Customized Init script

- MAC 주소 기반 IP 자동 할당
- SD카드 자동 포맷
- 부팅 시, 에러 발생 시 앱 자동 실행



Customized Rootfs 구축

- Qt5, ALSA, Telnetd 등 필수 패키지 포함
- 불필요한 파일 제거로 용량 최적화
- 한글 UI 완전 지원



RAM Disk vs SD카드 성능 비교

- 부팅 속도와 안정성을 고려한 설계 결정
- 루트 파일 시스템 -> RAM Disk
리소스 파일(이미지, 음악 등) -> SD카드

3. 핵심 기술: 커널 포팅

1. 임베디드 시스템 환경 구축 Booting Sequence

Setup Network

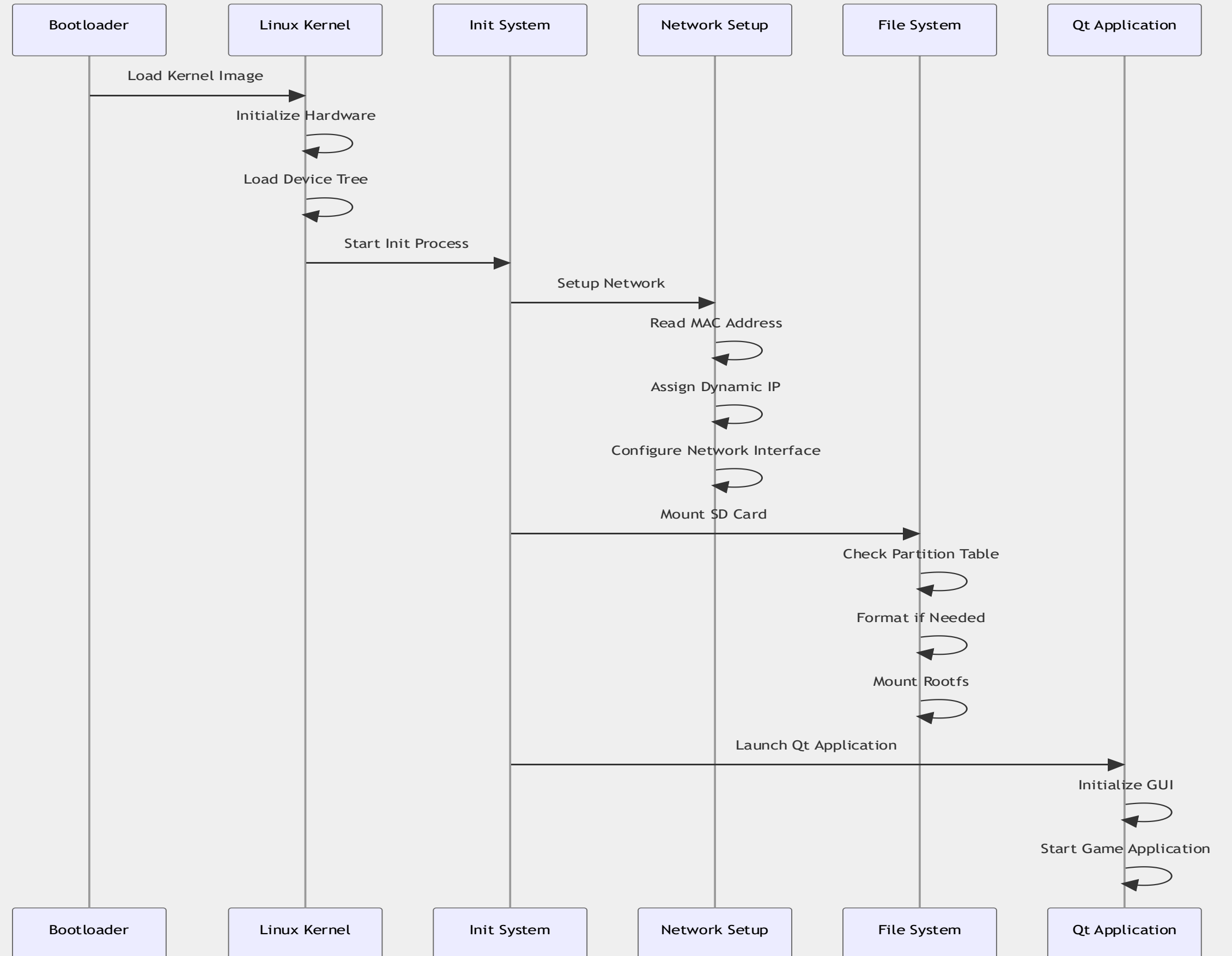
- 현재 보드의 MAC 주소 확인
- MAC 주소의 값에 따라 동적으로 IP 할당

Mount SD Card

- Partition table 확인
- (최초 1회) SD카드를 ext4 형식으로 포맷
- 루트 파일 시스템에 마운트

Launch Qt Application

- 필요 환경 변수 설정
- 부팅 후 App 자동 실행
- App 비정상 종료 시 다시 실행



3. 핵심 기술: 커널 포팅

1. 임베디드 시스템 환경 구축

Custom init script

set_ip_by_mac.sh

현재 보드의 MAC 주소 확인

MAC 주소의 값에 따라 동적으로 IP 할당

: 하나의 RFS Image로 여러 보드에 배포 가능

sd_partition_format.sh

Partition table 확인

(최초 1회) SD카드를 ext4 형식으로 포맷

: 각 보드에 추가적인 포맷 설정 불필요

```
$ set_ip_by_mac.sh
1  INTERFACE="eth0"
2
3  get_mac_address() {
4      cat /sys/class/net/$INTERFACE/address
5  }
6
7  assign_ip_by_mac() {
8      local mac=$1
9      case $mac in
10         "b8:27:eb:9d:e5:c8")
11             echo "192.168.10.4"
12             ;;
13         "b8:27:eb:9d:e5:c9")
14             echo "192.168.10.5"
15             ;;
16         "b8:27:eb:9d:e5:ca")
17             echo "192.168.10.6"
18             ;;
19         "b8:27:eb:9d:e5:cb")
20             echo "192.168.10.7"
21             ;;
22         "b8:27:eb:9d:e5:cc")
23             echo "192.168.10.8"
24             ;;
25         *)
26             echo "192.168.10.3 (default)"
27             ;;
28     esac
29 }
30
31 assign_ip() {
32     local ip=$1
33     ip addr flush dev $INTERFACE
34     ip addr add $ip/24 dev $INTERFACE
35     ip link set $INTERFACE up
36     echo "Assigned IP: $ip"
37 }
38
39 MAC_ADDRESS=$(get_mac_address)
40 echo "Detected MAC Address: $MAC_ADDRESS"
41
42 IP_ADDRESS=$(assign_ip_by_mac $MAC_ADDRESS)
43 assign_ip $IP_ADDRESS
44
```

```
sd_partition_format.sh
1  DEVICE="/dev/mmcblk0"
2  PARTITION1="${DEVICE}p1"
3  PARTITION2="${DEVICE}p2"
4
5  echo "Checking current partition layout..."
6  echo "Partition layout:"
7  fdisk -l $DEVICE
8
9  PARTITION_NUMBER=$(fdisk -l /dev/mmcblk0 | awk '/Device/{flag=1;next}flag' | wc -l)
10 echo "Number of partitions: $PARTITION_NUMBER"
11
12 if [ "$PARTITION_NUMBER" -ge 2 ]; then
13     echo "There are $PARTITION_NUMBER partitions. Formatting process canceled."
14     exit 1
15 fi
16
17 P1_ENDLBA=$(fdisk -l /dev/mmcblk0 | grep '/dev/mmcblk0p1' | awk '{print $5}')
18 echo "EndLBA of p1: $P1_ENDLBA"
19
20 START_SECTOR=$((P1_ENDLBA + 1))
21 echo "Calculated start sector for p2: $START_SECTOR"
22
23 echo "Creating new partition (p2)..."
24 (
25     echo n          # create new partition
26     echo p          # default partition
27     echo 2          # partition number
28     echo $START_SECTOR # start sector
29     echo            # end sector (default; use all reserve space)
30     echo w          # save modified info
31 ) | fdisk $DEVICE
32
33 echo "Reloading partition table..."
34 partprobe $DEVICE
35
36 echo "Creating ext4 filesystem on ${PARTITION2}..."
37 mkfs.ext4 -F $PARTITION2
38
39 echo "New partition formatting is completed!"
```

3. 핵심 기술: 커널 포팅

1. 임베디드 시스템 환경 구축

Custom init script

init_app.sh

필요 환경 변수 설정

부팅 후 App 자동 실행

App의 세그멘테이션 폴트 등

비정상 종료가 감지될 경우, 즉시 재실행

: 사용자 혼란 최소화, 유지보수 부담 감소

```
$ init_app.sh
1  export QT_QPA_FONTDIR="/usr/lib/fonts"
2  export QT_QPA_PLATFORM="linuxfb"
3  export LANG="ko_KR.UTF-8"
4
5  while true; do
6      echo "Starting app..."
7      /root/hello_world
8      EXIT_CODE=$?
9      if [ $EXIT_CODE -eq 0 ]; then
10         echo "Application exited normally with code $EXIT_CODE"
11         break
12     elif [ $EXIT_CODE -eq 137 ]; then
13         echo "Application is killed by signal code $EXIT_CODE, exit."
14         break
15     else
16         echo "Application crashed with code $EXIT_CODE, Restarting..."
17         sleep 1
18     fi
19 done
20
```

```
# my custom init script
::sysinit:/bin/sh /root/set_ip_by_mac.sh
::sysinit:/bin/mkdir /mnt/nfs
::sysinit:/bin/mount -t nfs -o nolock 192.168.10.2:/nfsroot /mnt/nfs
::sysinit:/bin/sh /root/sd_partition_format.sh
::sysinit:/bin/mkdir /mnt/sd
::sysinit:/bin/mount -t ext4 /dev/mmcblk0p2 /mnt/sd
::sysinit:/bin/ln -s /root/aplay /root/arecord

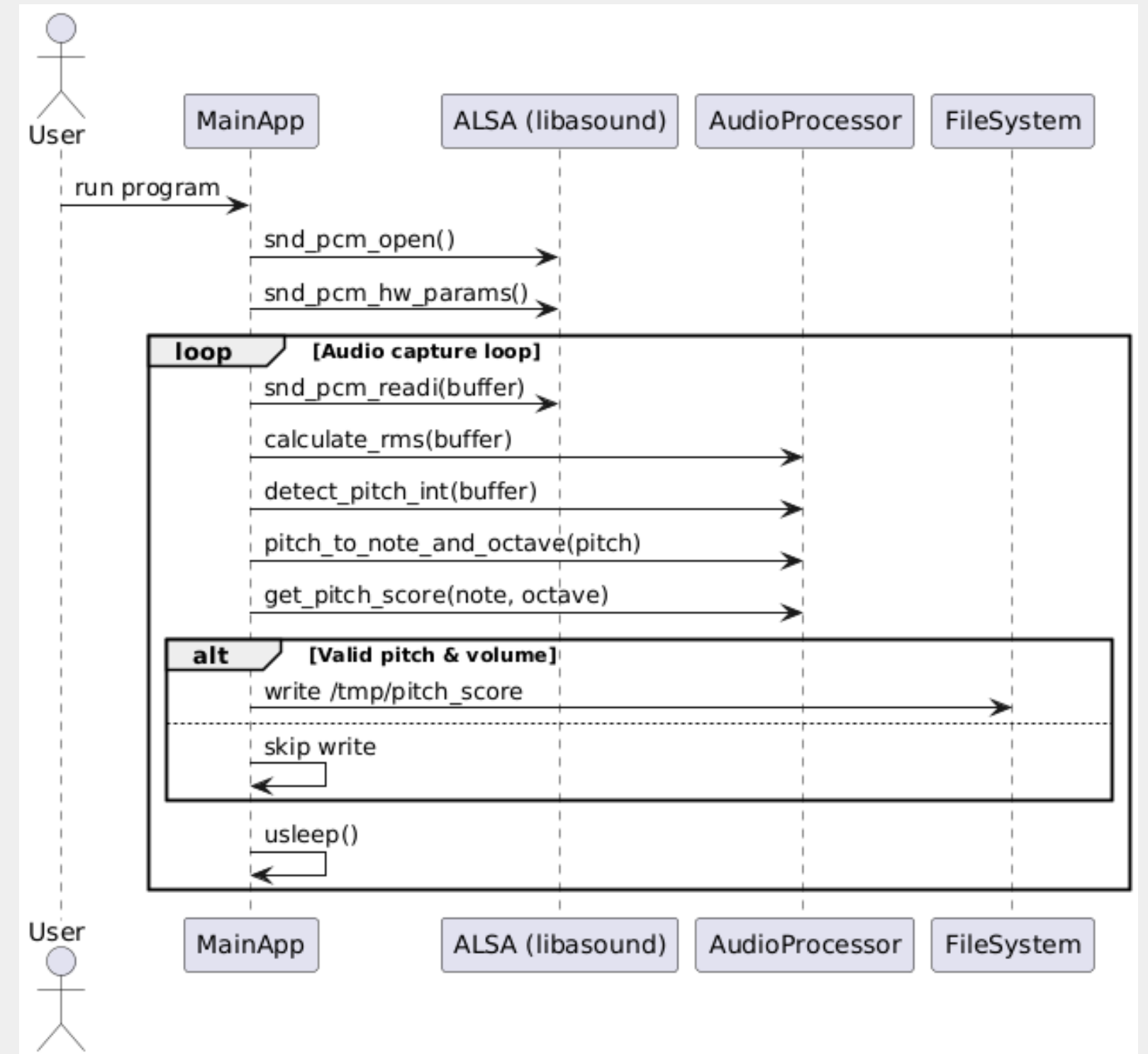
# app init
::sysinit:/bin/sh /root/init_app.sh
```

Custom init scrip를
/etc/inittab 작성하여 실행

3. 핵심 기술: 디바이스 드라이버

🎤 2. 실시간 오디오 입력 및 음정 분석

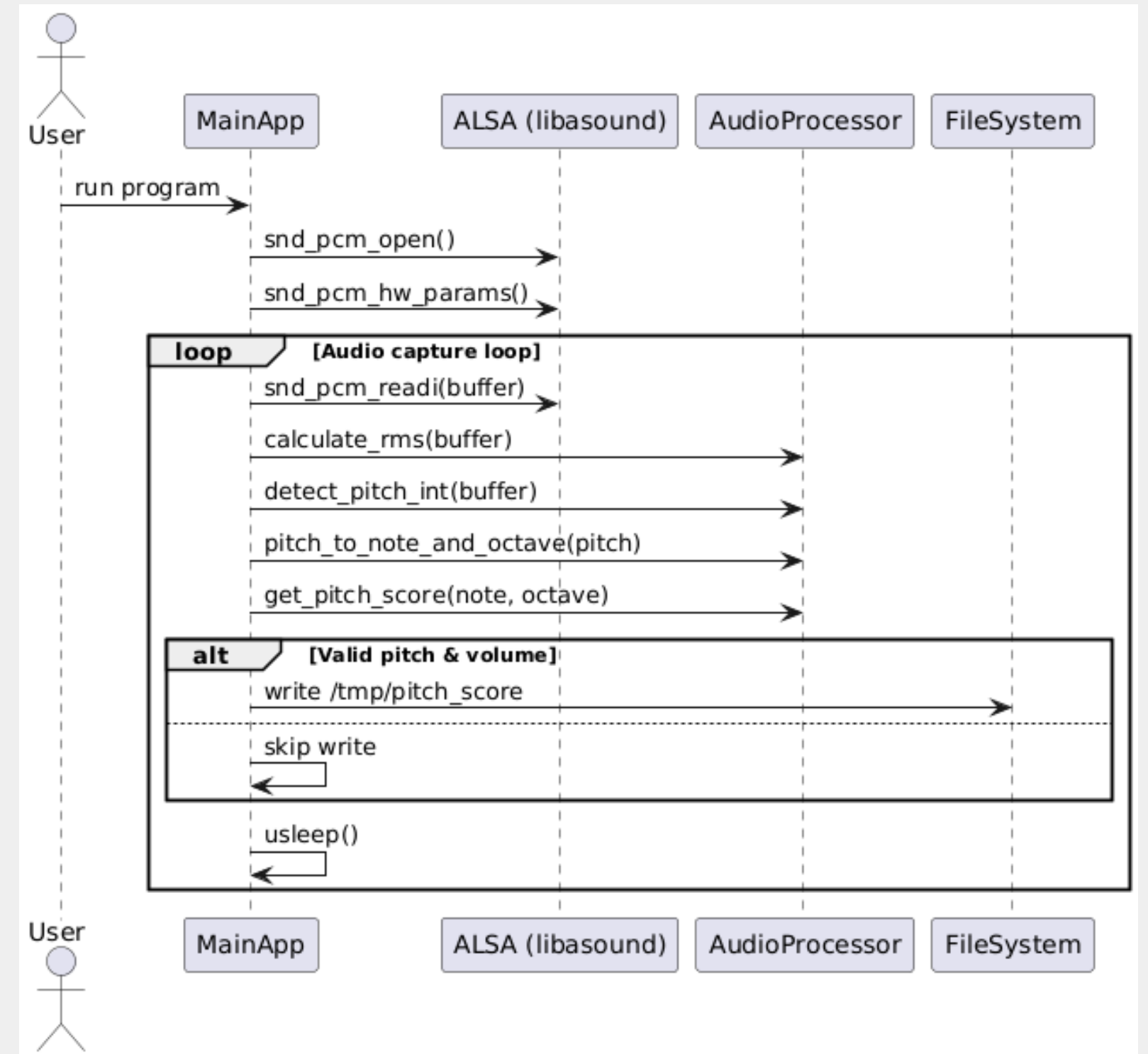
- ALSA(Audio Linux Sound Architecture)를 사용해 마이크로부터 실시간 PCM 오디오 데이터를 수집
- 입력 신호에서 RMS(Root Mean Square) 계산을 통해 볼륨 측정
- 일정 수준 이하의 신호는 배경 소음으로 판단해 무시함으로써, 사용자 음성만을 정확하게 인식할 수 있도록 설계



3. 핵심 기술: 디바이스 드라이버

🎤 2. 실시간 오디오 입력 및 음정 분석

- FFT(Fast Fourier Transform) 알고리즘을 활용해 시간 영역에서 기본 주파수(피치)를 추정
- 가장 큰 에너지 가지는 주파수 중심으로 양 옆 스펙트럼 값 이용, 선형 보간
- 추정된 피치를 MIDI 음정 값으로 변환, 이를 다시 게임 화면의 Y좌표에 매핑하여 캐릭터 위치 제어
- 사용자가 정확한 음정을 유지할수록 캐릭터가 장애물을 잘 피할 수 있음



3. 핵심 기술: APP

실시간 오디오 피드백

- ALSA 마이크 입력을 자동상관함수도 피치 검출하여 MIDI 음계를 게임 Y좌표도 매핑, 음성으로 실시간 캐릭터 제어

난이도 조절

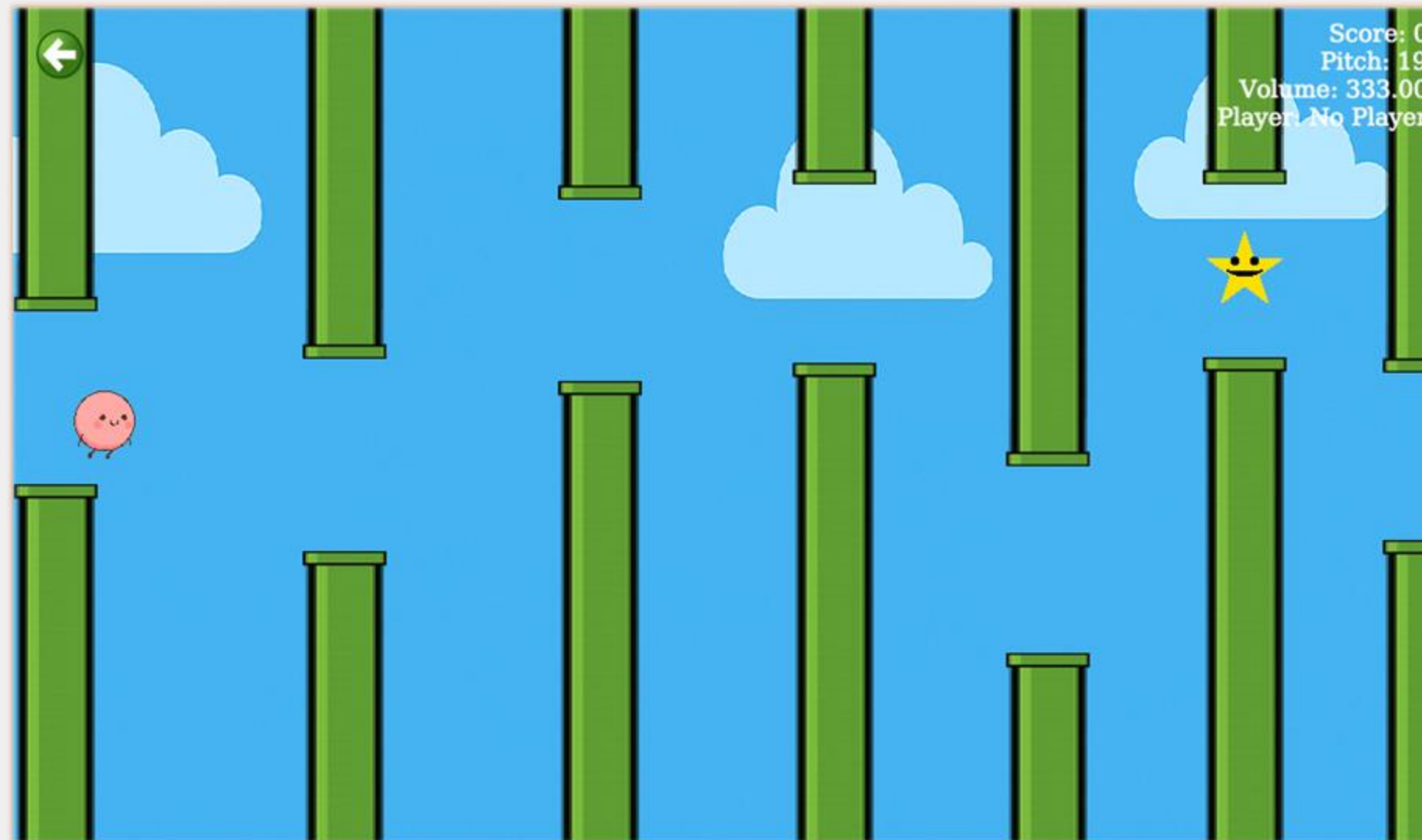
- 상하 장애물 간 gap 차이, 장애물 생성 주기 등 변수들은 구간마다 난수 생성하여 게임 난이도 설정

플레이어 관리

- 개별 플레이어 이름 설정으로 각 게임별 독립적인 랭킹 시스템 구현 및 점수 기록 관리

세가지 모드 지원

- Single Play, Multi Play, Sing Along 등 세가지 모드를 선택하여 즐기수 있는 게임의 폭은 넓힘



Sound System 제공 (배경음, 효과음)

- 배경음악, 효과음 등 스피커를 통해 즐거운 게임 환경을 생성

2인 이상 멀티미디어 모드 지원

- UDP 소켓 통신을 통한 여러 명 동시 게임 가능

랭킹시스템

- Single play에서 각 플레이어마다의 랭킹 기록 저장 가능

Qt를 활용한 UI 구현

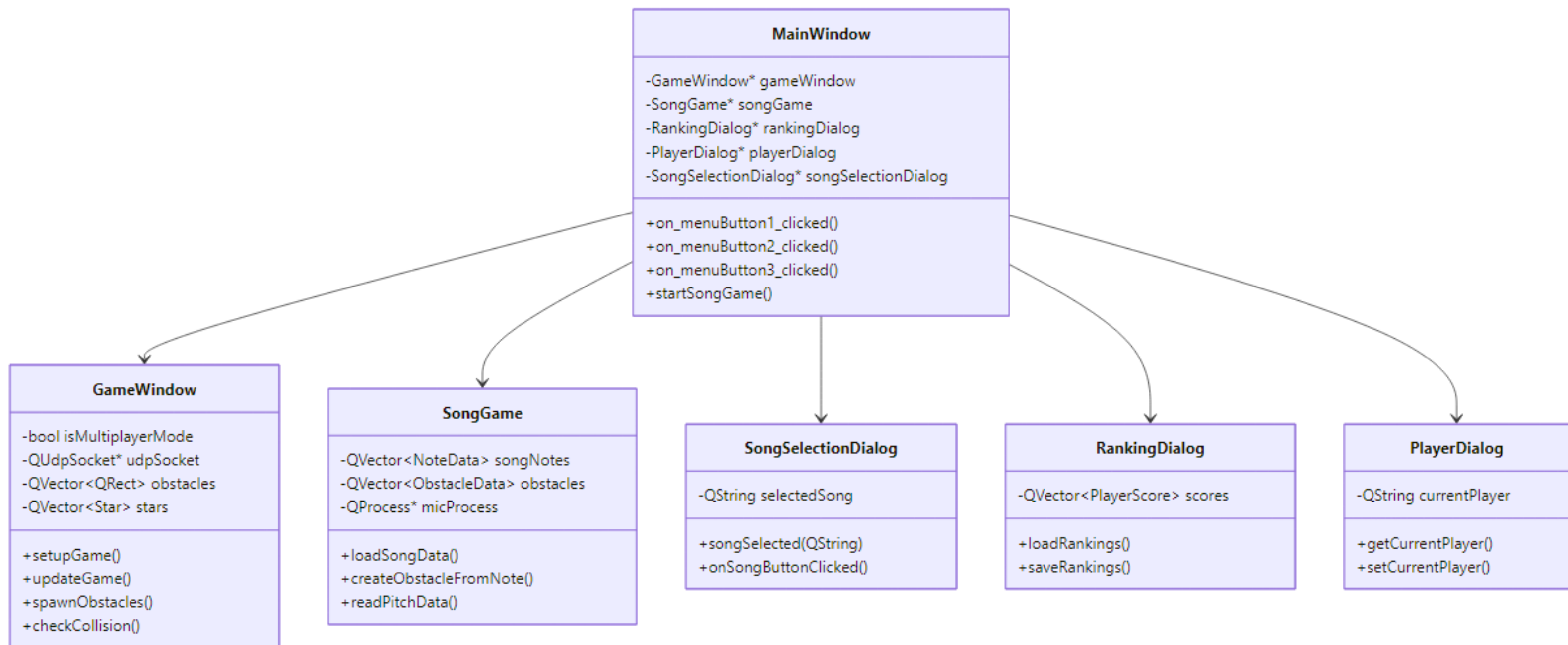
- Single play에서 각 플레이어마다의 랭킹 기록 저장 가능

Single Play

Multi Play

Sing Along

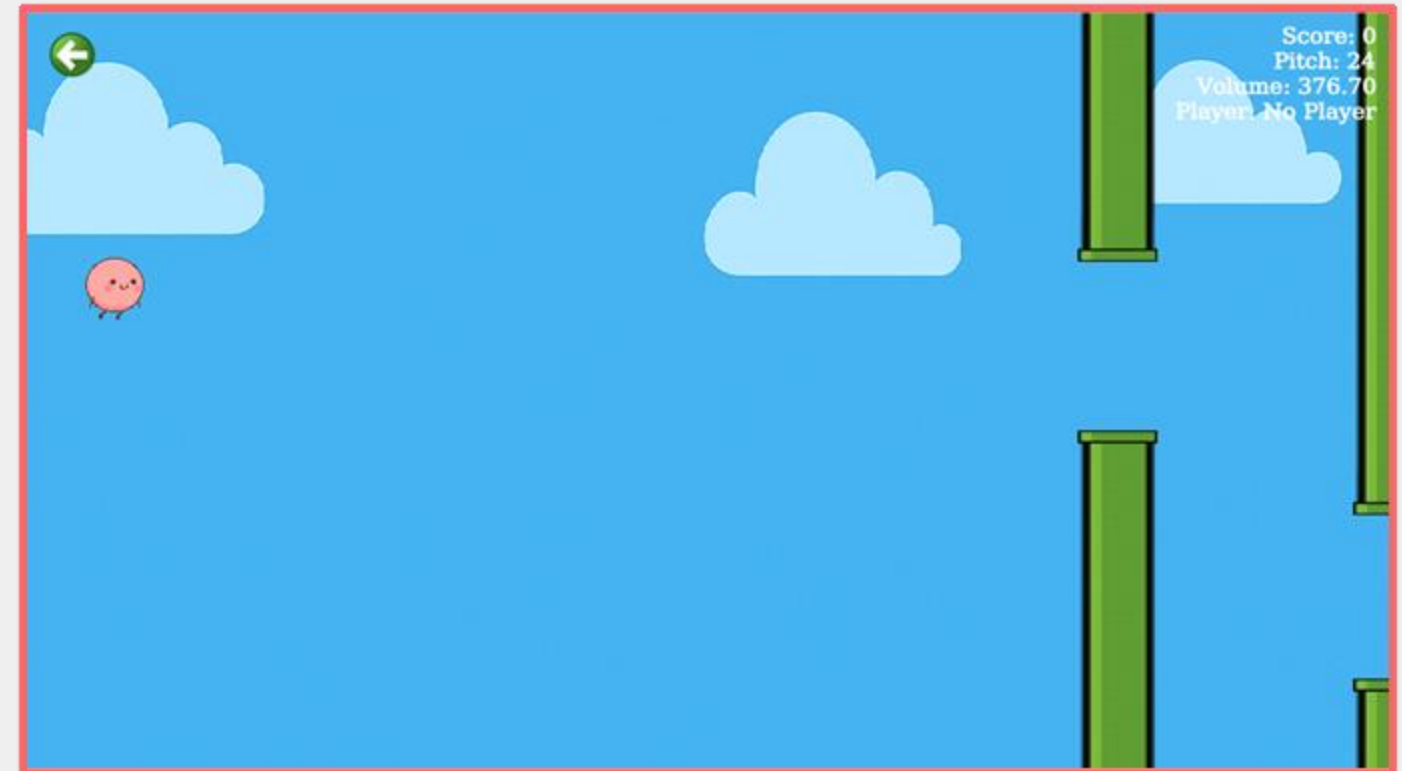
3. 핵심 기술: APP



3. 핵심 기술

📌 1. 게임 모드 구성 및 피드백 시스템

- 싱글 모드: 사용자의 음정에 따라 장애물을 피하며 점수 획득
- 멀티플레이어 모드: 여러 명이 동시에 플레이하며 경쟁 (UDP 기반 통신)
- Sing Along 모드:
 - CSV 포맷 노래 데이터를 파싱하여 박자/가사/음정 기반 장애물 자동 생성
 - 노래를 정확하게 부르면 캐릭터가 자연스럽게 전진하며 점수 누적
- 사용자가 직접 자신의 음정 정확도를 체험하며 피드백을 받을 수 있음



3. 핵심 기술: APP

🎵 2. Activity한 게임 요소추가

- 배경음악 및 효과음 시스템

ALSA 기반 리눅스 오디오 시스템 연동하여 음악 및 효과음 재생

배경음악의 경우 자동루프 재생 매커니즘을 추가하여 반복재생 적용

장애물에 충돌, 아이템 획득 시 효과음 추가하여 생동감 있는 게임 환경 구축

- 장애물에 따른 난이도 제어와 아이템 구현으로 재미요소 추가

점수 구간에 따른 장애물 간격 (최소,최대 지정) 및 장애물 생성 간격 조정으로 점진적 난이도 증가 구현

아이템은 획득 시 +3점 점수 추가를 통한 게임성 강화

- 플레이어 관리 및 랭킹시스템을 통해 경쟁 요소 도입

플레이어 등록 및 전환 기능을 통해 플레이어별 점수 기록 구현

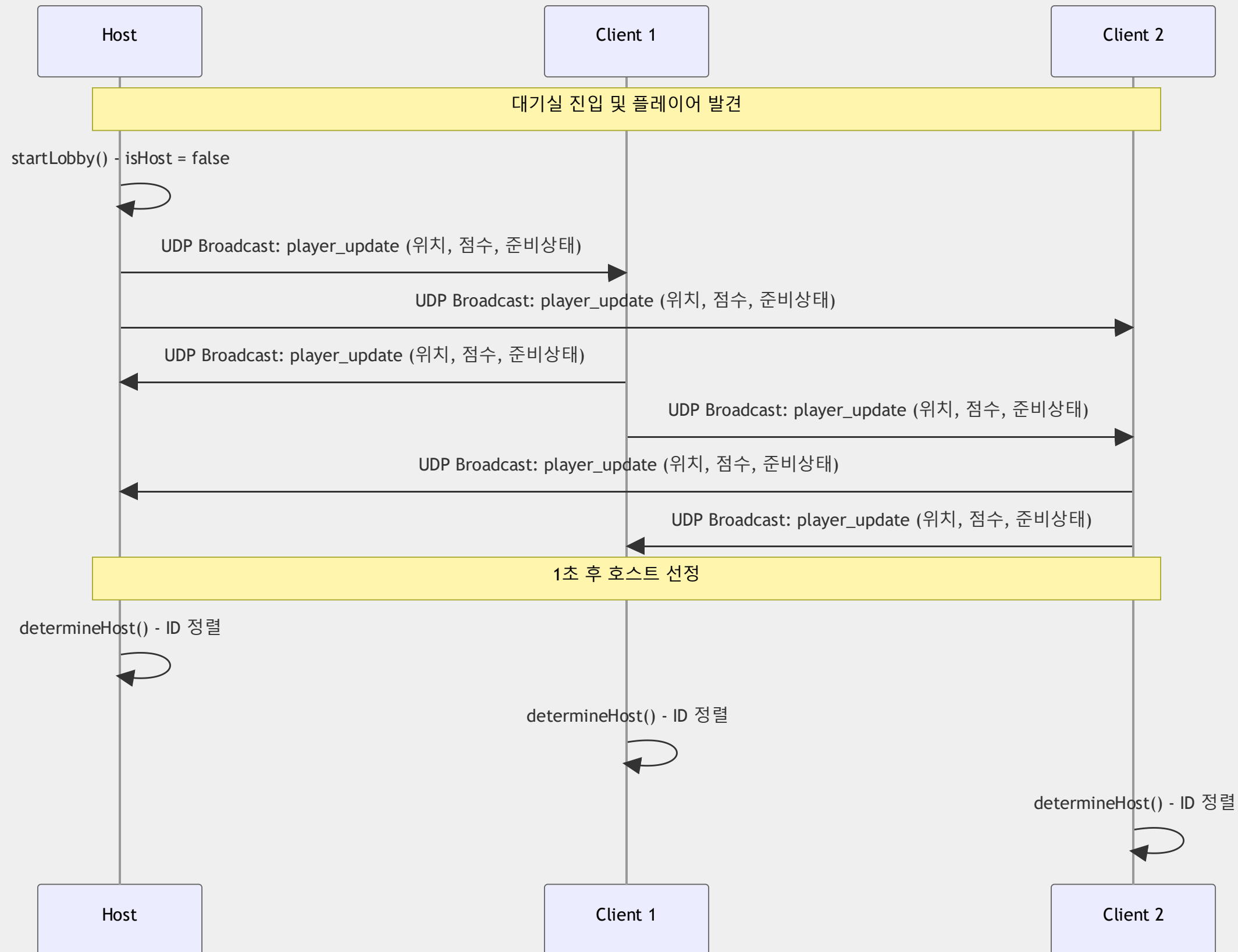
시각화된 랭킹 시스템 (1위~7위)으로 다른 플레이어의 기록 확인, 경쟁 심리 유발

3. 핵심 기술: APP

3. 네트워크 멀티플레이어 시스템

- UDP 멀티캐스트 통신 구조를 구현하여, 모든 클라이언트에게 실시간으로 게임 상태를 전송
- QJsonDocument + QUdpSocket을 활용해 JSON 형식의 데이터 직렬화/역직렬화
- 타임스탬프 기반의 지연 보정 로직을 구현해 네트워크 환경에서 발생하는 딜레이 최소화
- 충돌/장애물 위치/점수 정보 등을 실시간으로 동기화하여, 자연스러운 멀티플레이 경험 제공

3. 핵심 기술: APP



- 대기실 진입 시간 동안 무제한 동시 접속 지원
- 시드값 기반의 난수 맵 자동 생성 로직을 구현해
네트워크 패킷 전송량 최적화
- 충돌/장애물 위치/점수 정보 등을 실시간으로
동기화하여, 자연스러운 멀티플레이 경험 제공

3. 핵심 기술: APP

4. 성능 최적화

- 메모리 관리

Qt의 부모-자식 관계를 활용하여 객체의 생명주기를 관리하고, 객체 소멸 시 모든 리소스가 자동으로 해제

렌더링 최적화

배경 및 스프라이트 이미지를 캐싱하여 재사용함으로써 메모리 사용량을 줄이고 렌더링 성능을 향상

화면 밖에 있는 객체는 렌더링하지 않도록 하여 불필요한 그리기 작업을 제거

- 네트워크 최적화

고유 시드값은 동한 난수값 맵 생성으로 맵 정보는 실시간 동기화 되면서, 맵 정보는 전송 불필요하게 만들어 트래픽을 감소

타임스탬프를 사용하여 오래된 데이터는 무시하도록 하여 네트워크 지연에 의한 문제를 최소화

4. 결과 분석 및 기대 효과

✓ 목표 달성 평가

- 3가지 게임 모드(싱글, 멀티, 노래 연습) 구현 완료
- ALSA 기반 실시간 오디오 캡처와 Autocorrelation 피치 분석의 정확도와 실시간성 만족
- UDP 멀티캐스트 기반 네트워크 통신을 통한 멀티플레이어 동기화 지연 최소화 성공
- Buildroot 기반 임베디드 시스템에서 커널 포팅과 루트 파일 시스템 안정적 구동 및 자동 실행 달성

⚠ 한계 및 개선점

- 네트워크 환경 변화에 따른 지연과 패킷 손실 영향이 일부 발생
- 음정 검출 정확도 향상을 위해 노이즈 제거 및 신호 처리 고도화 필요
- 게임 UI 및 UX 일부 부분에서 사용자 피드백 반영한 개선점 존재

✨ 기대 효과

- 음악 교육용 게임으로서 사용자 음정 인식과 피드백 제공 가능
- 멀티플레이어 기능으로 친구나 그룹 단위 학습 시너지 기대
- 임베디드 환경에서의 안정적 동작으로 다양한 하드웨어 플랫폼에 적용 가능
- 향후 AI 평가 시스템 등 고도화 연구의 기반 마련

5. 향후 연구 과제

향후 연구 과제

- AI 기반 음성 정확도 평가 시스템 도입
 - 사용자 음성 자동 평가 및 피드백 고도화
- 소셜 기능 강화
 - 친구 추가, 채팅, 리더보드 확장으로 사용자 참여도 향상
- 다양한 곡목 자동 추가 및 관리 시스템 개발
 - CSV 외 다양한 포맷 지원, 곡 데이터베이스 구축
- 모바일 플랫폼(Android/iOS) 버전 포팅
 - 휴대기기에서의 사용성 확대 및 접근성 강화
- 네트워크 안정성 및 지연 보정 알고리즘 개선
 - 다양한 네트워크 환경에서 원활한 멀티플레이어 경험 제공

6. 프로젝트 수행 후기

프로젝트 수행 후기

- ALSA 오디오 처리부터 커널 포팅, GUI, 네트워크까지 다양한 기술을 한 프로젝트에서 경험
- 실시간 음성 인식과 멀티플레이어 동기화 구현이 가장 어려웠지만, 문제 해결 과정에서 큰 성취감
- Buildroot와 임베디드 시스템 설정을 통해 하드웨어와 소프트웨어 통합 이해도 상승
- 팀 내 협업과 테스트 반복을 통해 소통 능력 및 개발 프로세스 개선 경험
- 앞으로도 복합 시스템 개발 역량을 강화하고 싶다는 동기 부여가 된 프로젝트